PROGRAM 10 – A* SEARCH ALGORITHM

AIM

To write a Python program to implement the A* Search algorithm to find the shortest path between two nodes in a graph.

ALGORITHM

1. Start the program.
2. Create a graph with nodes and edge costs.
3. Define heuristic values for each node.
4. Read the starting node and goal node.
5. Initialize a priority queue.
6. Calculate the evaluation function using \(f(n)=g(n)+h(n)\).
7. Select the node with the lowest evaluation value.
8. Expand its neighbouring nodes.
9. Update the path cost and parent information.
10. Repeat until the goal node is reached.
11. Display the shortest path and total cost.

CODE

```
import heapq
graph={"A":{"B":1,"C":4},"B":{"D":2,"E":5},"C":{"E":1},"D":{"G":3},"E":{"G":2},"G":{}}
heuristic={"A":6,"B":4,"C":3,"D":3,"E":1,"G":0}
def astar(start,goal):
    queue=[(heuristic[start],0,start,[start])]
    visited={}
    while queue:
        f,g,node,path=heapq.heappop(queue)
        if node==goal:
            return path,g
        if node in visited and visited[node]<=g:
            continue
        visited[node]=g
        for neighbor,cost in graph[node].items():
            new_g=g+cost
            new_f=new_g+heuristic[neighbor]
```

```
            heapq.heappush(queue,(new_f,new_g,neighbor,path+[neighbor]))
    return None,None
start=input("Enter starting node: ").upper()
goal=input("Enter goal node: ").upper()
if start in graph and goal in graph:
    path,cost=astar(start,goal)
    if path:
        print("Shortest Path:"," -> ".join(path))
        print("Total Cost:",cost)
    else:
        print("Path not found")
else:
    print("Invalid node")
```

INPUT

```
Enter starting node: A
Enter goal node: G
```

OUTPUT

```
Shortest Path: A -> B -> D -> G
Total Cost: 6
```

RESULT

Thus, the A* Search algorithm was successfully implemented in Python to find the shortest path using path cost and heuristic values.